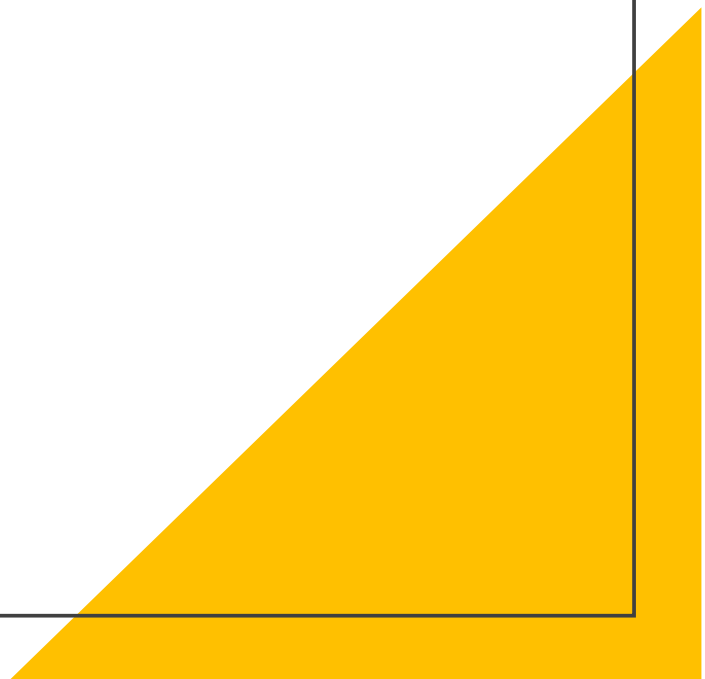


串接Ai



OpenAI

要怎麼讓自己的網頁有辦法串接AI功能其實很簡單，不外乎就是看自己要使用哪個AI工具就去看他的文件，正常情況下會是在後端處理這部分(因為安全性問題)，這邊我們會教各位怎麼串接OpenAI的。

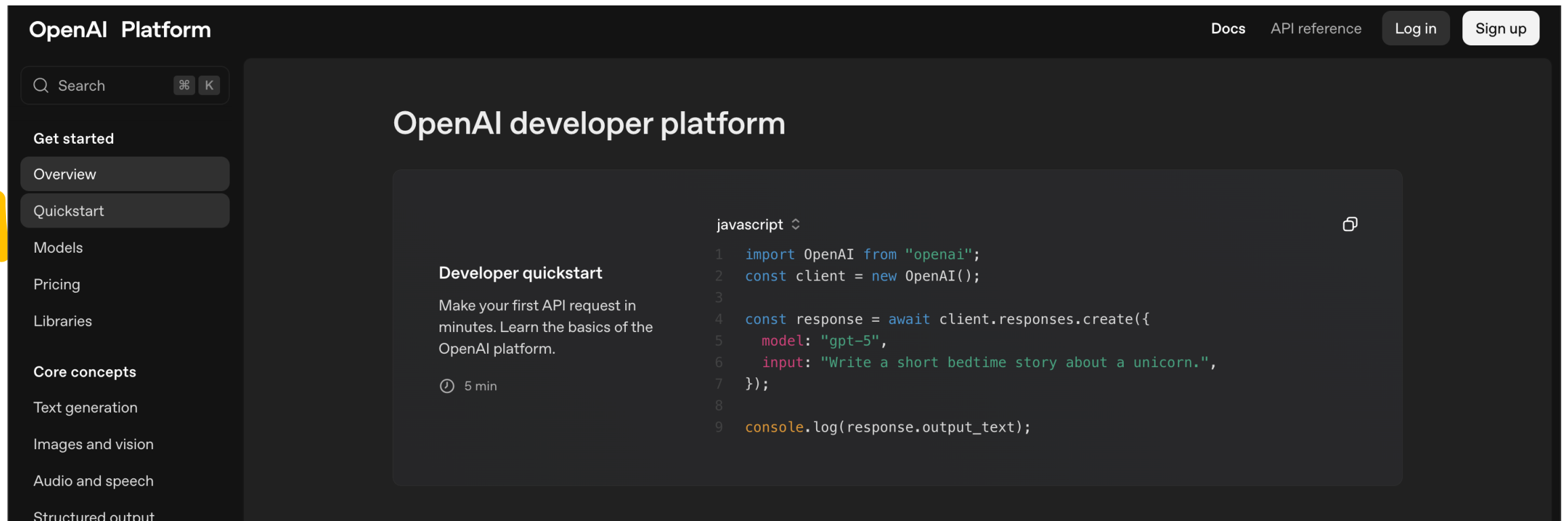
第一步 需要先去申請呼叫AI的Api的金鑰

第二部 在網站上寫呼叫Api的程式跟畫面

第三部 呈現Api回應給你的內容

申請OpenAI的金鑰

首先先來到這個網頁並且登入<https://platform.openai.com/docs/overview>。

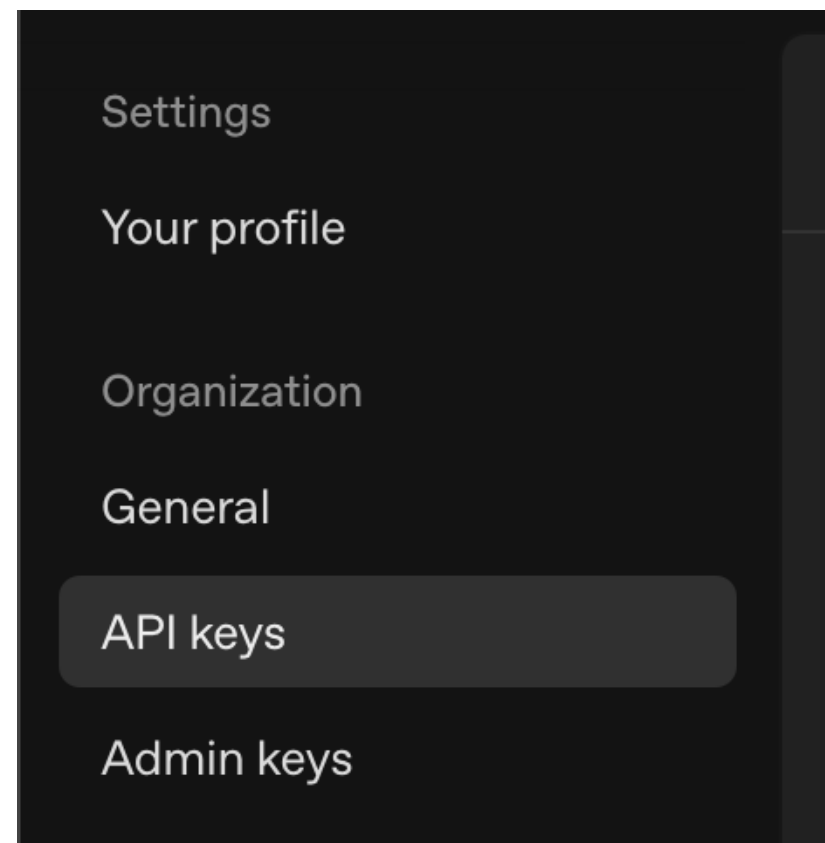
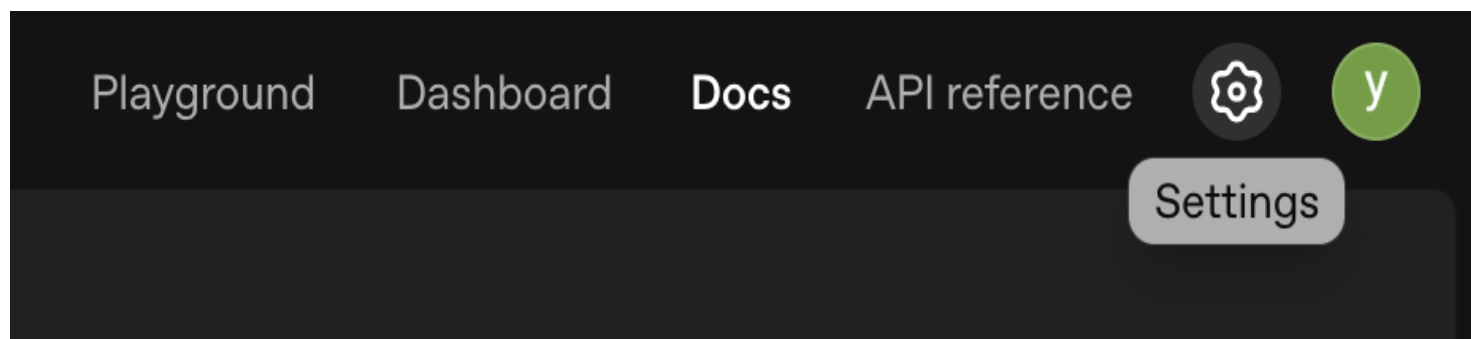


The screenshot shows the OpenAI Platform developer quickstart page. The left sidebar contains a navigation menu with the following items: **OpenAI Platform**, a search bar, **Get started** (with sub-items Overview and Quickstart), **Models**, **Pricing**, **Libraries**, **Core concepts** (with sub-items Text generation, Images and vision, Audio and speech, and Structured output). The main content area is titled "OpenAI developer platform" and features a "Developer quickstart" section. This section includes the text "Make your first API request in minutes. Learn the basics of the OpenAI platform." and a clock icon indicating a 5-minute duration. To the right of this text is a code block for a JavaScript example, which includes a copy icon. The code is as follows:

```
javascript ↕
1 import OpenAI from "openai";
2 const client = new OpenAI();
3
4 const response = await client.responses.create({
5   model: "gpt-5",
6   input: "Write a short bedtime story about a unicorn.",
7 });
8
9 console.log(response.output_text);
```

申請OpenAI的金鑰

然後點擊頭像隔壁的齒輪，然後左邊選單的API Keys。



申請OpenAI的金鑰

來到這個畫面點選新增一個key。

API keys

+ Create new secret key

You have permission to view and manage all API keys in this organization.

Do not share your API key with others or expose it in the browser or other client-side code. To protect your account's security, OpenAI may automatically disable any API key that has leaked publicly.

View usage per API key on the [Usage page](#).

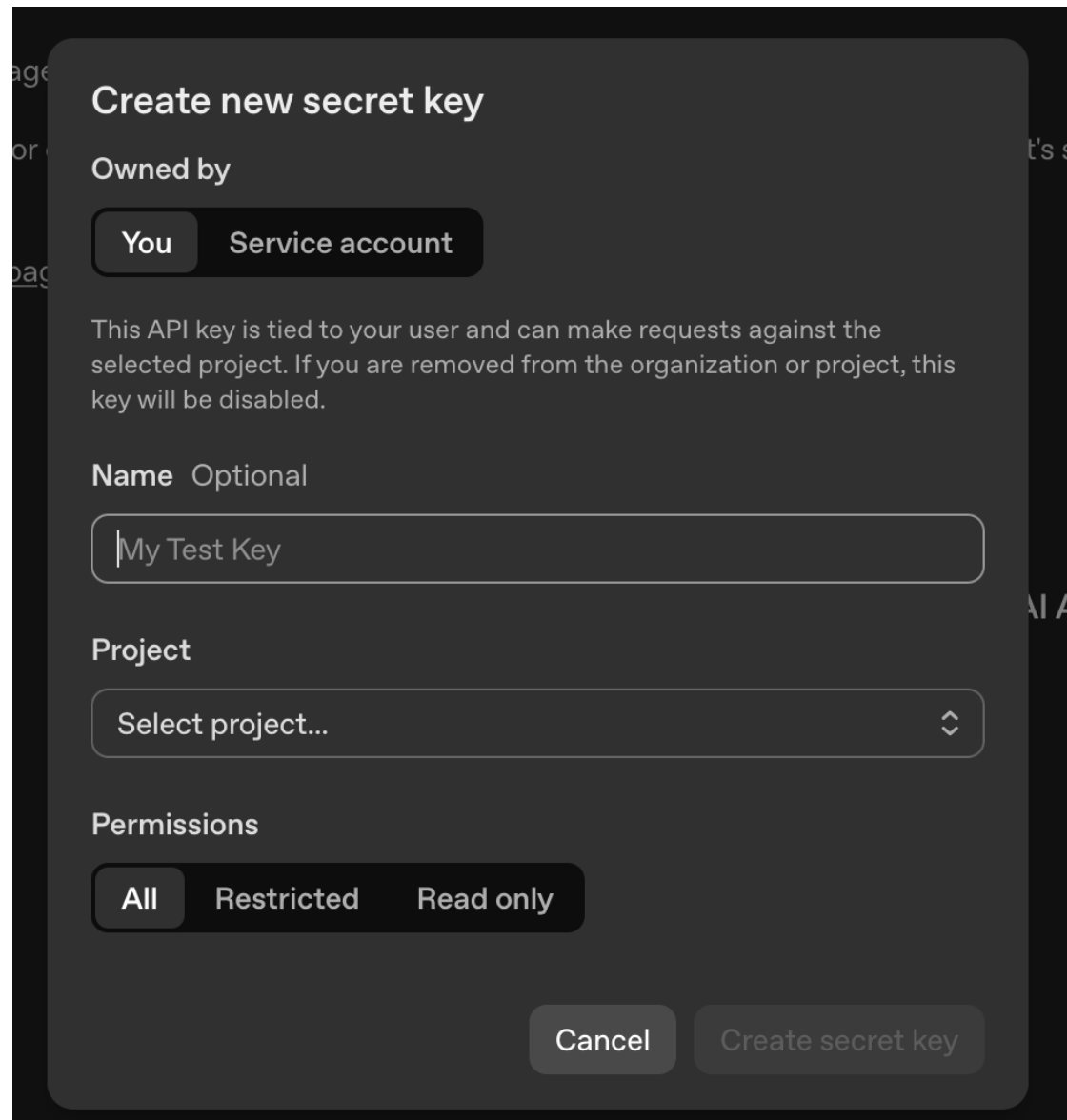


Create an API key to access the OpenAI API

+ Create new secret key

申請OpenAI的金鑰

然後會來到這個畫面取名並且生成專案選預設，然後把KEY複製起來存起來，後面呼叫API需要使用。



The screenshot shows the 'Create new secret key' dialog box in the OpenAI interface. It has a dark theme. At the top, it says 'Create new secret key'. Below that, 'Owned by' has two tabs: 'You' (selected) and 'Service account'. A warning message states: 'This API key is tied to your user and can make requests against the selected project. If you are removed from the organization or project, this key will be disabled.' The 'Name' field is optional and contains 'My Test Key'. The 'Project' dropdown menu is set to 'Select project...'. Under 'Permissions', the 'All' tab is selected, with 'Restricted' and 'Read only' as alternatives. At the bottom right are 'Cancel' and 'Create secret key' buttons.

Create new secret key

Owned by

You Service account

This API key is tied to your user and can make requests against the selected project. If you are removed from the organization or project, this key will be disabled.

Name Optional

My Test Key

Project

Select project...

Permissions

All Restricted Read only

Cancel Create secret key

寫Api文件

接著我們就要來寫Api文件，跟之前教的一樣使用httpClient，所以我們先在之前建立的service.ts裡面新增兩個全域變數用來存放Api位置跟金鑰的值。

```
// api的網址  
apiUrl = 'https://api.openai.com/v1/chat/completions';  
  
// 你的金鑰  
OPENAI_API_KEY = "sk-proj-ruBdAzUs-SloKW9S3YFH8v_xWq1dwqbZQtYmTI8oUDH3ok5BXGjJl1PYd9khLjjv39M0RpxdcWT3BlbkFJ3  
constructor(private httpClient: HttpClient) { }
```

http.service.ts

寫Api文件

接著新增一個方法用來呼叫Ai的Api。

```
callOpenAi(msg: string) {  
  // 呼叫Api的時候要帶過去的抬頭  
  const headers = new HttpHeaders({  
    'Content-Type': 'application/json',  
    'Authorization': `Bearer ${this.OPENAI_API_KEY}`  
  });  
  
  // 呼叫Api的時候要帶過去的內容  
  const body = {  
    model: 'gpt-4o-mini', // 你可以改成 gpt-4o 或 gpt-3.5-turbo  
    messages: [{ role: 'user', content: msg }]  
  };  
  
  // 因為要傳遞內容過去所以使用post  
  return this.httpClient.post(this.apiUrl, body, { headers });  
}
```

http.service.ts

呼叫Ai Api

接著在你要用使用Api的文件裡面呼叫這個方法(我在app.component.ts中呼叫)，我這邊是寫在ngOnInit裡面，可以試著把它改成當使用者輸入內容按發送的時候再去呼叫這隻Api並且把輸入的內容取代妳好。

```
ngOnInit(): void {  
  this.httpService.callOpenAi('妳好')  
    .subscribe((res: any) => {  
    console.log(res.choices[0].message.content);  
  })  
}
```

app.component.ts

结束

